

Zielsetzung

Die letzte Aufgabe für dieses Semester in STEP bestand darin eine dynamische Webanwendung zu erstellen, wo die von *rhodes* empfangenen Daten angezeigt werden. Zudem sollen die Daten der jeweiligen Schiffe eine eigene Seite bekommen, welche auf der Hauptseite mit der MMSI des Schiffs verlinkt ist. Eine Sicherung der projekt-relevanten Skripte sowie die Erstellung eines Posters mit $\text{\LaTeX}$ waren ebenfalls Teile des Projekts. Durch das erlernte Wissen aus STEP konnte das Projekt erfolgreich abgeschlossen werden. Von *cat*, *mv*, *cd*, *ls* zu *VAAK*, *Schleifen* und der *Pipe* wurde nahezu alles genutzt, um das Projekt zu realisieren.

Ida Rhodes (*1900 †1986) war eine Computerpionierin, welche als eine der ersten das Potenzial der Informationstechnik gesehen hat. In ihrem Vortrag '[t]he human computer's dream of the future' erzählte sie von ihrer Überzeugung der Nutzung dieser im Alltag [(1)]. Ihr zu Ehren wurde der automatische Identifikationssystem (kurz AIS) -Empfänger, welcher an der Hochschule genutzt wird, mit dem Namen *rhodes* versehen.

Webanwendung

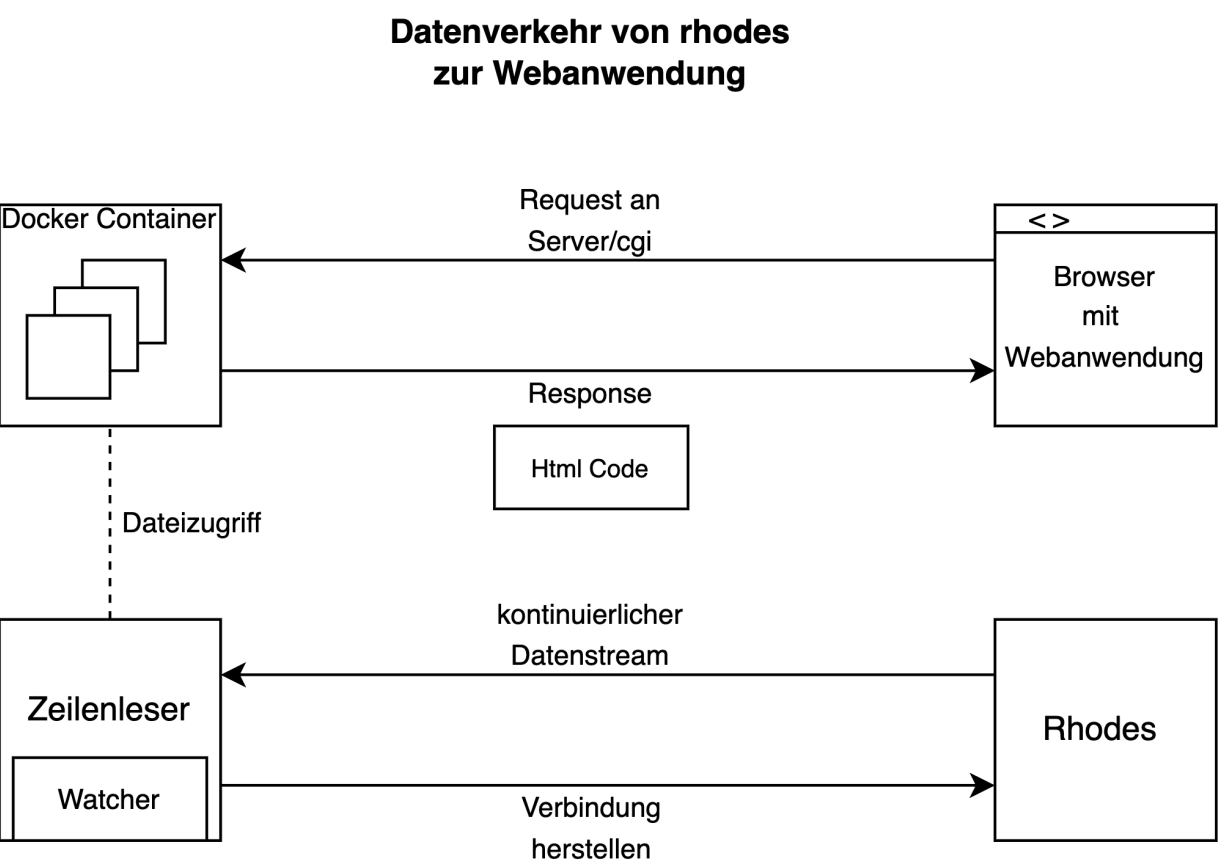


Abbildung 1: Interaktion Webanwendung

mmsi.sh

Als erstes wurde der Zeilenleser gebaut, um Daten von *rhodes* abzurufen und diese in eine csv-Datei umzuleiten. Danach wurde die Umgebung der Anwendung erstellt. Hierfür wurden Vorlagen für das Html-Gerüst (head, end) entwickelt worden, um diese immer wieder abrufen zu können. Dann wurde die Tabelle geöffnet, in der die Daten aufgeführt werden sollen. Die Daten sind mit *cat* aus der csv-Datei gezogen worden und mit *cut*, *uniq* und *sort* bearbeitet. Durch eine *while-read-Schleife* sind die Daten in die Tabelle eingefügt worden. Hiernach ist die Nummer der MMSI als Verlinkung genutzt worden, um es in einem weiteren Skript als *QUERY_STRING* benutzen zu können. Anschließend ist die Tabelle geschlossen und am Ende der Html durch die vorgefertigte Text-Datei mit *cat* ausgegeben worden.

result.sh

Anfangs wird wieder das Html-Gerüst genutzt, um die Seite zu beginnen. Die Variable 'schiff' wurde mit *Kommandosubstitution* initialisiert, in der der *QUERY_STRING* mit *cut* aufgeteilt wurde, um so *nur* die Nummer der MMSI zu deklarieren. Danach wurde die *while-read-schleife* mit den Daten, die der Zeilenleser über *rhodes* empfangen hat, erstellt und teilt diese in vier 'Spalten' auf, wobei '|' als Trennzeichen genutzt wurde. Danach wurde die Variable 'mmsi' mit der Variable 'schiff' durch eine *if-Bedingung* verglichen. Sofern diese erfüllt ist werden die gelesenen Daten in eine Tablle eingefügt. Zum Schluss wurde die Tabelle dann geschlossen. Wie auch schon wie im vorherigen Skript wurde die Html mit der Vorlage beendet.

Skripte

Watcher

Mit dem Watcher kann geprüft werden, ob die Webanwendung stabil ausgeführt wird und, sofern notwendig, neu gestartet werden kann. Zu Beginn wurde mit der Initialisierung der Variable 'last' und 'now' mit *Kommandosubstitution* sowie bei 'now' mit *wc -l* gearbeitet. Danach wurde die Datei 'last.txt' mit 'now' überschrieben. Dies ermöglicht die erneute Ausführung der folgenden *if-Bedingung*. In der *if-Bedingung* wurde die Variable 'last' mit dem aktuellsten Stand der ausgelesenen Daten von *rhodes* in 'now' verglichen. Wenn die Zeilenanzahl gleich ist, wird der Prozess zum Auslesen von *rhodes* abgebrochen und neu gestartet. Dabei wird dann die Prozess-ID in eine Variable 'ncatpid' gespeichert und mit *echo* in eine weitere Datei gespeichert, um es bei erneuten Vorkommen wiederholen zu können. Schließlich beendet sich der Prozess des Watchers von selbst.

Sicherung

Die 'aktuelle-version' der Anwendung wird in ein tar-Archiv gebündelt und in ein erstelltes Verzeichnis für die vorherigen Versionen der Anwendung verschoben.

```
1 ls -d -p *.tar | grep -v /
2 echo "gib den name des gewünschten backups
   ↳ ein oder nichts, falls keins gewünscht
   ↳ ist:"
3 read backup
4 if test -f "$backup"; then
5 echo "stelle wieder her: $backup"
6 mkdir backup_$backup
7 tar -xvf "$backup"
8 mv aktuelle-version/* ./backup_$backup
9 rm -r aktuelle-version
10 echo "Wiederherstellung fertig"
```

Listing 1: Wiederherstellung Backup

Dann beginnt die mögliche Wiederherstellung (s. Listing 1) einer vorherigen Version. Mit *ls* werden bestehende tar-Archive angezeigt und fordern dann den User auf, wenn gewünscht, ein Backup anzugeben. Wenn dies der Fall ist und eins angegeben wurde, wird mit einer *if-Bedingung* getestet, ob ein solches Archiv existiert. Ist die Bedingung erfüllt, wird ein Verzeichnis mit den Namen des Backups erstellt, bevor es selber entpackt wird. Die Dateien der enpackten Versionen werden dann in das erstellte Verzeichnis verschoben und das somit leere 'aktuelle-version'-Verzeichnis entfernt.

Lasttest

Beginnend mit der Initianlisierung von 'i' wird die erste *while-Schleife* gebaut. In dieser befindet sich eine zweite *while-read-Schleife* mit der Iteration 'i'. Innerhalb dieser wird die Webanwendung gecurlt mit der initialisierten Variable 'j' anstelle der MMSI. Dies wird umgelenkt in /dev/null und in den Hintergrund gelegt. Dann wird die Variable 'pid' des gerade gestarteten *curls* festgelegt. Danach wird die Überprüfung der Auslastungswerte von Docker begonnen, indem die letzte Zeile mit *tail -n 1* ausgegeben und die Trennzeichen durch ein Semikolon ersetzt werden. Die Werte werden dann umgeleitet in eine csv-Datei. Anschließend wird gewartet bis der aktuelle Curl-Prozess beendet ist und 'i' um eins erhöht. Zuletzt werden die Inhalte der csv-Datei nummeriert und das Semikolon dafür als Trennzeichen festgelegt. Führende Leerzeichen werden mithilfe von *sed* gelöscht und schließlich wird dies dann in eine entgültige csv-Datei geschrieben. Die Auslastung am Beispiel von 50 Iterationen wurde dann mit dem Programm *Gnuplot* visualisiert.

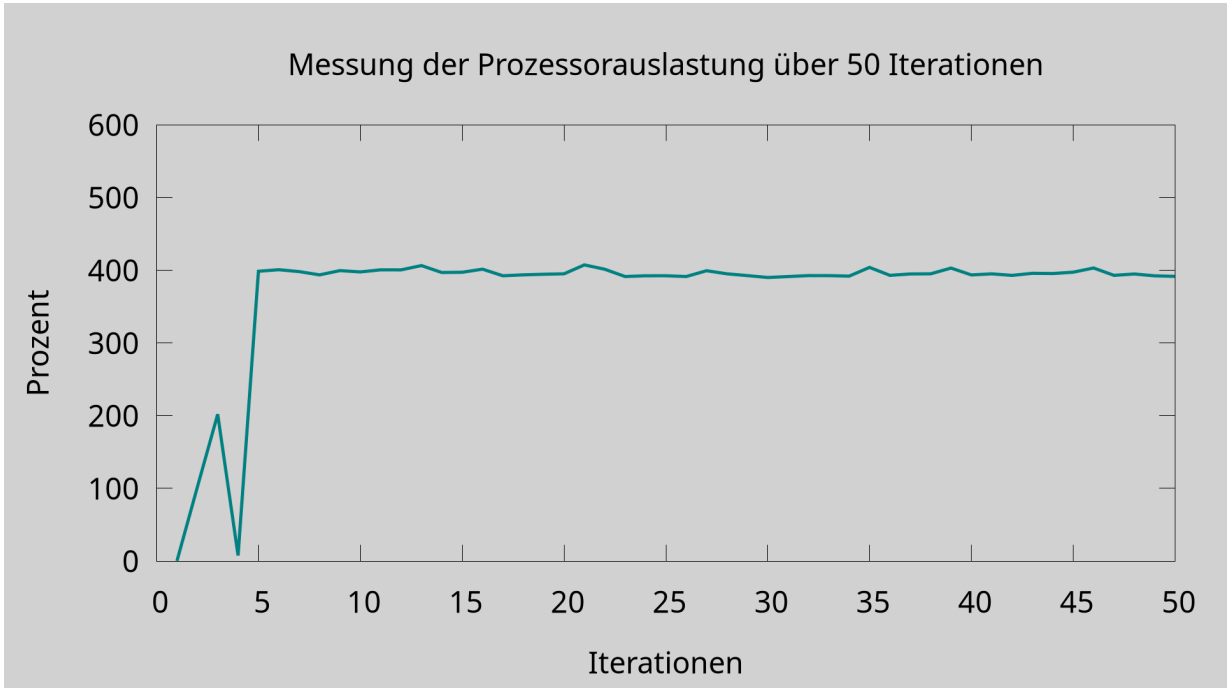


Abbildung 2: Lasttest 50 Iterationen

Referenzen

[1] V. Oechtering u. a. „Ida Rhodes.“ de, besucht am 10. Feb. 2025. Adresse: <https://www.frauen-informatik-geschichte.de/index.php-id=65.htm>.